

Sphatik OS — Engineering Documentation

Sep 27, 2026 · @Sarvesh Kumar Yadav

Overview and how to use these docs

This is the engineering handbook for building Sphatik OS: what to build, in what order, with which tools, on the Redmi Note 9 Pro (India model, codename curtana). The product and UI design lives in the separate [design specification](#); the working UI reference is the [shell prototype](#).

What Sphatik is, technically: a Linux-kernel phone OS with its own init, system services, Wayland compositor, UI toolkit, shell and apps, all written by us (mostly in Rust), plus an Android compatibility container so Play Store apps run alongside native ones.

Guiding engineering rules

1. **Own the experience, reuse the plumbing.** Everything the user sees is ours. Hardware plumbing (kernel, Mesa, ModemManager, PipeWire) is proven open source we integrate, not rewrite.
2. **Rust by default** for everything we write; C only where an upstream API forces it.
3. **Desktop first, device second.** Every component must run in a window on a Linux laptop before it runs on the phone, so the edit-run loop stays under a minute.
4. **Budgets are features.** The performance budgets in the design spec (60 fps, 750 MB idle RAM on 4 GB) are acceptance criteria, checked in CI.
5. **Private by default.** No telemetry, no Google services shipped in the image, permission prompts for every sensitive capability.

Glossary

Term	Meaning
curtana	Codename of the India Redmi Note 9 Pro; part of the miatoll device family (SM7125, Snapdragon 720G)
Mainline	The upstream Linux kernel from kernel.org, as opposed to Xiaomi's Android vendor kernel
Compositor	The program that owns the screen, draws every window, runs animations and handles touch
Shell	The system UI: lock screen, home, Halo, panels, switcher; runs inside the compositor

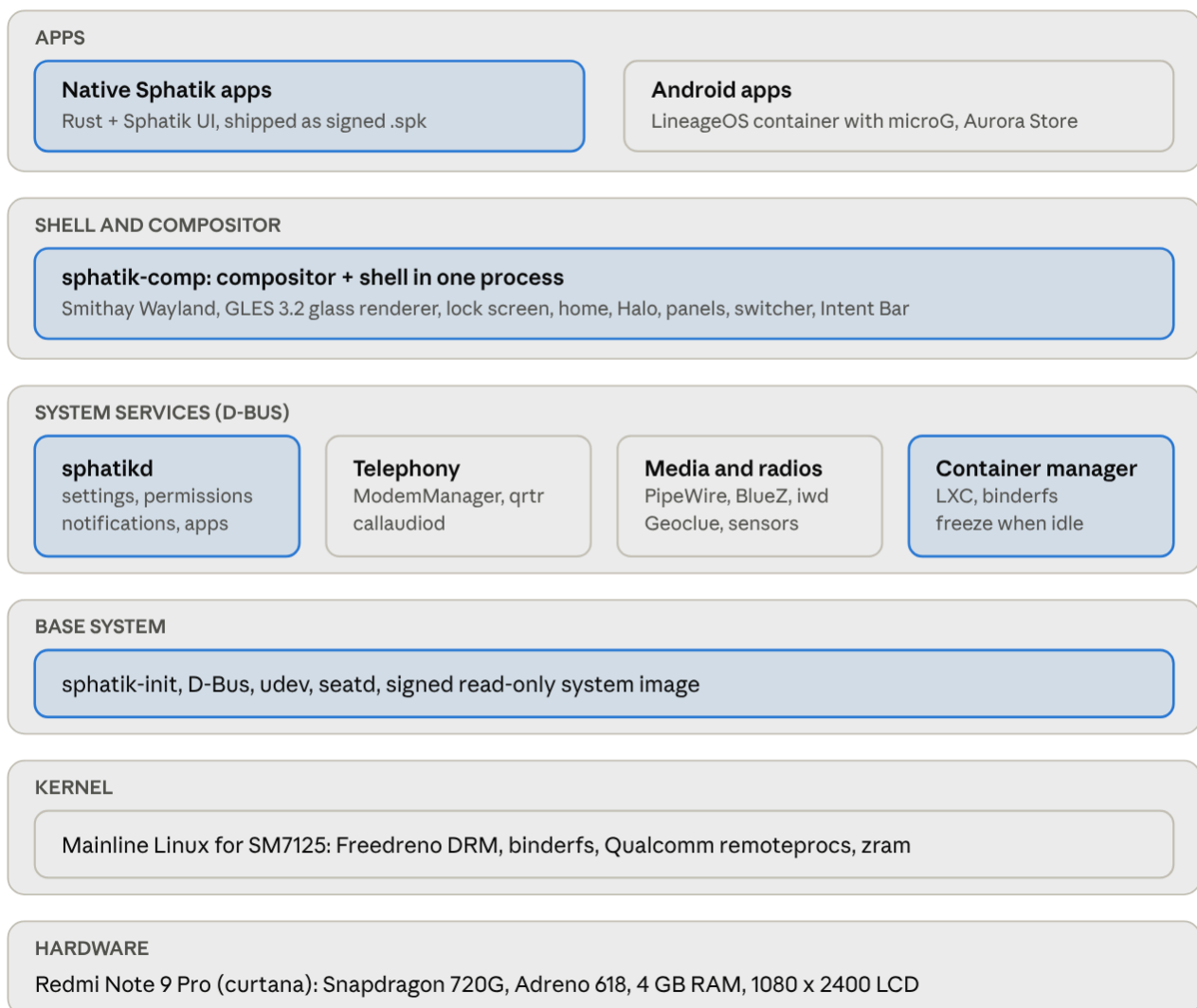
Term	Meaning
Container	The isolated Android system (Waydroid-style) that runs Android apps on Sphatik's kernel
microG	Open-source replacement for Google Play Services used inside the container
.spk	Sphatik package: the signed format for native apps

System architecture

Sphatik is six layers. We write the parts that define the experience and integrate mature open source for hardware plumbing.

Sphatik owns every layer the user sees; proven plumbing sits underneath

☒ Written by Sphatik ☐ Upstream open source we integrate



Sphatik OS layers · from hardware to apps

Process model

Process	Runs as	Role
sphatik-init	PID 1	Mounts filesystems, starts and supervises every service, handles shutdown and A/B boot success marking
sphatik-comp	Own user, real-time priority on a big core	Owens DRM/KMS and input, composites all windows, draws the shell and glass
sphatikd	System service	Settings store, permission broker, notification server, app install and launch, live-activity registry
ModemManager, callaudiod, PipeWire, BlueZ, iwd, Geoclue, iio-sensor-proxy	System services	Upstream hardware plumbing, reached over D-Bus
Container manager	Root helper + unprivileged LXC container	Starts, freezes and stops the Android container on demand
Apps	One sandboxed process each	Wayland clients; talk to the system only through sphatikd and portals

How the pieces talk

- **Wayland** between apps and the compositor (standard protocols plus private `sphatik_*` extensions for glass regions, live activities and the Halo).
- **D-Bus** between the shell, apps and system services (Rust side uses the `zbus` crate).
- **Binder** only inside the Android container.

Life of a touch, end to end: the touch panel raises an interrupt, the kernel emits an evdev event, libinput in sphatik-comp turns it into a gesture or a Wayland touch event, the shell or the app updates state, and the compositor renders the next frame within the 16.6 ms budget and scans it out on the next vblank.

Repository layout and build environment

One monorepo, one Cargo workspace, one command-line tool (`sp`) for every build, flash and debug task.

Repository layout

```
sphatik/  
├ docs/                design spec exports, this handbook, ADRs  
                        (architecture decision records)  
├ tools/sp/            the sp CLI: build, image, flash, logs, perf  
├ device/xiaomi-curtana/ kernel config fragment, firmware list, ALSA UCM,  
                        device tree notes  
├ system/  
│   ├── init/          sphatik-init (PID 1)  
│   └─ sphatikd/        settings, permissions, notifications, app manager  
├ compositor/          sphatik-comp (Smithay) and the glass renderer  
├ shell/               lock, home, library, panels, Halo, switcher, Intent  
Bar  
├ ui/                  sphatik-ui toolkit (widgets, layout, text, motion)  
├ apps/                phone, messages, camera, ... (one crate per app)  
├ sdk/                 app SDK crates, spk packager, templates  
├ android/             container image recipe, sphatik-bridge APK  
├ image/               system image recipe, initramfs, A/B layout  
└─ tests/              integration and on-device test suites
```

Host machine: a Linux laptop or desktop (Ubuntu 24.04 or Arch), x86_64, 16 GB RAM recommended, 100 GB free disk, USB cable for the phone.

Toolchain

Tool	Why
rustup, Rust stable, target <code>aarch64-unknown-linux-musl</code>	All Sphatik code; musl matches the Alpine-based system image
cargo-zigbuild	Simple cross-compiling and linking for aarch64 without a full sysroot
pmbootstrap	Builds the postmarketOS base image during device bring-up
android-tools (fastboot, adb)	Flashing the phone; debugging the Android container
qemu-user-static, binfmt	Running aarch64 binaries and tests on the laptop

Tool	Why
Mesa, libinput, libseat, libxkbcommon, libudev (dev packages)	Running the compositor in a window on the laptop
Android Studio (optional)	Building the sphatik-bridge APK

Everyday commands

```

cargo run -p sphatik-comp -- --windowed window      # compositor + shell in a laptop
cargo run -p app-calculator windowed compositor      # a single app against the
sp build --target curtana the phone                  # cross-compile everything for
sp image --device curtana image                     # assemble boot.img and system
sp flash userdata                                   # fastboot flash boot and
sp logs -f sphatik-comp over USB                    # stream logs from the phone
sp perf trace --seconds 10 device                   # frame-time trace from the

```

Continuous integration (runs on every pull request)

1. `cargo fmt --check` and `cargo clippy -D warnings`.
2. Unit and integration tests on x86_64.
3. Cross-build for aarch64.
4. Headless compositor tests with software rendering (golden screenshots of key screens).
5. Nightly: full image build and a performance run on a device connected to a self-hosted runner.

Working rules: `main` is always buildable; work in short-lived branches; conventional commit messages; every significant technical decision gets a one-page ADR in `docs/adr/`.

Kernel and device bring-up

We start from the community postmarketOS port for the miatoll family, which already boots a mainline kernel on SM7125, and replace its userspace with Sphatik step by step.

Current hardware status on mainline (from the community port; recheck on your unit)

Works	Broken or missing	Fixable
Display, touch, GPU, Wi-Fi, calls, mobile data, audio, sensors	Fingerprint, NFC, hardware video decode	Camera (community kernel patch)

Required kernel configuration (kept as a fragment in `device/xiaomi-curtana/kernel.config`)

Area	Options	Needed for
Graphics	DRM_MSM (Freedreno), DRM panel driver for the device	Compositor
Android container	ANDROID_BINDER_IPC, ANDROID_BINDERFS, namespaces, cgroup v2, PSI	Android apps
App sandbox	SECCOMP_FILTER, SECURITY_LANDLOCK, USER_NS	Native app isolation
Memory	ZRAM, ZSMALLOC, MEMCG	4 GB RAM budget
Storage and integrity	EROFS or SQUASHFS, DM_VERITY, DM_CRYPT or FS_ENCRYPTION	Read-only system image, encrypted data
Qualcomm plumbing	QRTR, QCOM remoteproc (modem, ADSP, CDSP), RMTFS memory	Modem, audio DSP, sensors
Debug	USB gadget (CDC NCM), ftrace, BPF	SSH over USB, performance tracing

Firmware: modem, ADSP, CDSP, GPU zap shader and Wi-Fi firmware are closed binaries from the stock MIUI firmware partitions. The build extracts them from the user's own phone at install time; Sphatik does not redistribute them. Userspace helpers `rmtfs`, `pd-mapper`, `tqftpserv` and the QRTR name service must run before the modem and DSPs come up.

Bring-up procedure

1. **Back up** everything; unlocking wipes the phone. Use a spare phone as your daily driver.
2. **Flash the latest stock MIUI fastboot ROM** so all firmware partitions are consistent.
3. **Unlock the bootloader** with Xiaomi's Mi Unlock tool (it enforces a waiting period).
4. **Build the base image:** `pmbootstrap init` → vendor xiaomi, device miatoll → `pmbootstrap install`.

5. **Flash** with fastboot as the port's instructions describe (boot image plus root filesystem on userdata).
6. **Run the hardware checklist** below and record results in `device/xiaomi-curtana/STATUS.md`.
7. **Swap in Sphatik userspace** one layer at a time: compositor first (replacing Phosh), then sphatikd, then sphatik-init.

Hardware checklist: screen on at 60 Hz; touch with 10 points; GPU via `glmark2-es2`; Wi-Fi scan and connect; SIM detected; outgoing and incoming call with audio both ways; SMS send and receive; mobile data; speaker, earpiece, headphones, mic; Bluetooth audio; accelerometer and proximity; charging and battery percentage; suspend and resume.

Known gotchas from the port: the Plymouth boot splash hangs (Sphatik draws its own splash from the compositor); the boot image must fit the small cache partition; the battery gauge reports nonsense for the first seconds (ignored for 10 s by sphatikd).

Staying safe: never flash or erase `persist`, `modem` or `efs`-style partitions; fastboot mode is always reachable with Volume Down + Power; the stock ROM can be restored with Xiaomi's flashing tool.

Upstream first: kernel fixes go to the device maintainers and mainline; packaging fixes go to postmarketOS. Carrying private patches is the most expensive thing a small OS team can do.

Boot, init and system services

From power button to lock screen in under 15 seconds, with a signed read-only system and user data that stays encrypted until the first unlock.

Boot sequence

1. **Xiaomi bootloader** loads `boot.img`: kernel, device tree and the Sphatik initramfs.
2. **Initramfs** (a small Rust program) draws the SKY splash straight to the display, picks system slot A or B using a boot counter, verifies the slot's dm-verity root hash against Sphatik's public key, and mounts it read-only.
3. **switch_root** hands over to `sphatik-init` as PID 1.
4. **sphatik-init** starts services in four stages: early (udev, Qualcomm helpers, zram), core (D-Bus, sphatikd, PipeWire, ModemManager, iwd), UI (sphatik-comp, which keeps the splash on screen and cross-fades to the lock screen), and lazy (Android container manager, started on first use).
5. **After 60 seconds of healthy uptime** init marks the slot as good; three failed boots roll back to the other slot automatically.

Filesystem layout

Path	Storage	Contents
<code>/</code>	EROFS image, read-only, verified	The whole OS: binaries, libraries, default config, fonts, wallpapers
<code>/var</code>	ext4 or f2fs on userdata	System state, logs, caches, installed-app registry
<code>/home/user</code>	fscrypt, key derived from the PIN	Personal files, app data; locked until the first unlock after boot
<code>/apps</code>	userdata	Installed native apps (.spk contents)
<code>/android</code>	userdata	Android system image, container data, shared folder

sphatik-init

- Service files are small TOML descriptions: command, user, dependencies, restart policy, cgroup memory limit, allowed capabilities.
- Every service gets its own cgroup, so memory limits and freezing work per service.
- Crash policy: restart with back-off; the compositor restarts within 1 second and restores the session.
- Kept deliberately small: no network, timers or logging built in; those are separate services.

sphatikd D-Bus interfaces

Interface	What it does
<code>org.sphatik.Settings</code>	Typed settings store with change signals
<code>org.sphatik.Permissions</code>	Checks, prompts, grants, auto-revoke, 7-day access log for the privacy dashboard
<code>org.sphatik.Notifications</code>	Receives, groups, schedules summaries, applies Focus rules
<code>org.sphatik.Apps</code>	Install, update, uninstall and launch native and Android apps
<code>org.sphatik.LiveActivities</code>	Registry that feeds the Halo (priority: call, navigation, timer, media, downloads)

Interface	What it does
<code>org.sphatik.Power</code>	Battery, charging, thermal state, Battery saver, suspend policy

Power management: screen off enters suspend-to-idle within seconds; wake sources are the modem (calls, SMS), the RTC (alarms, timers) and the power button. CPU frequency uses the `schedutil` governor; the UI thread and compositor are pinned to the two big cores, background work to the little ones.

Logging: every service writes structured logs to a small ring-buffer log daemon; `sp logs` streams them over USB. Crash reports stay on the device unless the user shares them.

Compositor and graphics

`sphatik-comp` is a Wayland compositor built on the Smithay library. It owns the display and touch input, draws every app window, and renders the shell and all glass in one GPU pass per frame.

Why Wayland and Smithay: Wayland gives us a standard, secure protocol for apps (and it is what the Android container already speaks), Smithay gives us a Rust foundation for DRM/KMS, input and rendering, and we keep full control of the look.

Backends

Where	Display	Input	Renderer
Phone	DRM/KMS + GBM + EGL on Freedreno	libinput via libseat	GL ES 3.2 with Sphatik shaders
Laptop	winit window	Mouse and keyboard, touch simulated by mouse	Same GL ES renderer

A Vulkan renderer (Turnip driver) is a later option; GL ES first keeps the path simple and well supported on Adreno 618.

Frame loop: frames are driven by vblank at 60 Hz. The compositor tracks damage so unchanged regions are not redrawn, keeps full-screen video on a hardware plane when possible (direct scan-out), and aims to finish composition in under 8 ms, leaving headroom within the 16.6 ms frame.

Glass render pipeline (per glass region, per frame)

1. Copy the pixels behind the region from the already-composited layers below.
2. Downsample to a quarter resolution.

3. Dual Kawase blur, 2 to 4 passes depending on the material.
4. Upsample and refract: a signed-distance field of the rounded shape bends the backdrop inward near the edges.
5. Tint with the adaptive colour; raise tint automatically if text contrast drops below 4.5:1.
6. Rim light and a specular spot positioned from the gyroscope.
7. A 0.5 px chromatic split on the outer rim only.
8. **Caching:** if nothing behind a panel changed, reuse last frame's blurred result. This is what keeps the Balanced tier under 3 ms of GPU time.

Wayland protocols

Kind	Protocols
Core app support	wl_compositor, xdg-shell, linux-dmabuf, viewporter, fractional-scale, presentation-time
Input and keyboard	wl_seat (touch), text-input-v3, input-method-v2, virtual-keyboard
Security	security-context-v1 (marks sandboxed apps), no screen capture without a portal prompt
Sphatik private	<code>sphatik_glass_v1</code> : an app declares rounded regions and a material, and the compositor draws real glass behind them. <code>sphatik_shell_v1</code> : privileged surfaces (keyboard, system dialogs)

Window model: one full-screen window per app on the phone; the app switcher shows each app's last frame kept as a GPU texture; split screen and floating windows arrive in v2 as extra layout modes, not a different window system.

Scaling: the 1080 x 2400 panel runs at a scale of 2.75, giving the 393-point-wide logical canvas the design spec uses; apps receive exact fractional scale so text stays crisp.

Gestures: edge swipes (home, back, panels) are recognised by the compositor before any app sees the touch, using the 20 px edge zones from the spec; everything else is forwarded to the app under the finger.

Keyboard: a separate `sphatik-keyboard` process, attached through input-method-v2, so it can be replaced by third-party keyboards like any other app.

UI toolkit and shell

`sphatik-ui` is the one toolkit used by the shell, the stock apps and third-party native apps, so every screen shares the same components, motion and glass.

Building blocks

Concern	Choice	Notes
Programming model	Declarative views over reactive state (signals)	Rebuild only what changed; no virtual DOM diffing every frame
Layout	<code>taffy</code> (flexbox and grid)	Same mental model as the prototype's CSS
Text	<code>cosmic-text</code> with <code>rustybuzz</code> shaping	Devanagari and other Indic scripts from day one; glyph atlas on the GPU
App rendering	GPU 2D canvas on GLES (<code>femtovg</code> to start)	Apps draw into their own buffers; glass comes from the compositor via <code>sphatik_glass_v1</code>
Shell rendering	Directly in the compositor's renderer	Lets the shell read the backdrop for true glass without extra copies
Accessibility	<code>AccessKit</code> tree for every widget	Feeds the Sphatik screen reader and automated UI tests
Strings	Project Fluent	English and Hindi first; plural and gender rules handled properly

Motion: one spring engine shared by shell and apps, with the four presets from the design spec (Snappy, Smooth, Bouncy, Gentle). Animations are driven by the compositor's frame callbacks, are interruptible at any frame, and inherit finger velocity when a gesture ends.

Theming: design tokens (surface, on-surface, accent, glass-tint, glass-rim and the rest) come from a single theme file. On wallpaper change, `sphatikd` extracts a palette and publishes new token values; every running app updates live.

Component library: glass title bar, floating tab bar, search field, segmented control, toggle, slider, list row and grouped card, avatar, floating action button, pushed view with back navigation, bottom sheet, menu, toast and banner, text field, date picker and empty state. The shell prototype is the visual reference for each one.

Shell modules (one Rust module each, in `shell/`)

Module	Maps to prototype screen	Data from
boot	SKY splash	Initramfs hand-off
lockscreen	Lock screen, customisation sheet	Settings, Notifications, Power
home	Home grid, widgets, edit mode, dock	Apps, Settings
library	App Library	Apps
panels	Notification Center, Control Center	Notifications, Settings, NetworkManager-style services
halo	Halo live activities	LiveActivities
switcher	App switcher, gesture navigation	Compositor window list
intent	Intent Bar search and answers	Apps, Settings, local index

UI performance rules: no heap allocation in per-frame paths; text layout cached until content changes; images decoded on a worker thread; every list virtualised beyond 50 rows; any frame over 12 ms of CPU time is logged with its call stack in debug builds.

App platform and SDK

Native Sphatik apps are Rust programs built with `sphatik-ui`, packaged as signed `.spk` files, and run in a sandbox that can only reach the system through permission-checked interfaces.

Package format (.spk)

Part	Contents
<code>manifest.toml</code>	App id (reverse domain, e.g. <code>in.sphatik.notes</code>), name, version, icon layers, permissions requested, intents offered, widgets and live-activity templates
<code>bin/</code>	The aarch64 executable (musl, statically linked where practical)
<code>res/</code>	Icons, images, Fluent string files, fonts
<code>SIGNATURE</code>	Ed25519 signature over all of the above by the developer's key

The archive is `tar` compressed with `zstd` . Sphatik checks the signature on install and on every update; an update must be signed by the same key as the installed app.

Sandbox: each app runs as its own Linux user in its own mount, PID and network namespaces, with a seccomp system-call filter and Landlock file rules. It sees only its own data folder, the files the user picks through a portal, and the Wayland and D-Bus sockets. Network access is a permission like any other.

Permissions: declared in the manifest, requested at the moment of use, granted by the user in a system prompt drawn by the shell (apps cannot fake it). Options and auto-revoke rules follow the permission table in the design spec. Every access is logged for the privacy dashboard.

Portals (the only doors out of the sandbox)

Portal	Gives the app
File picker	One-time access to files or folders the user chooses
Camera, microphone	A stream, with the status-bar privacy dot shown by the system
Location	Precise or approximate position, while-using or one time
Share sheet	Send content to other apps, contacts or nearby devices
Notifications	Post notifications to a declared channel
Contacts, calendar	Selected entries or all, as granted

Extension points for app developers

- **Intents:** declare actions such as `message.send` or `ride.book` with typed parameters; the Intent Bar can then offer them.
- **Widgets:** declare sizes and a render function; widgets update at most every 15 minutes unless showing a live activity.
- **Live activities:** Compact text, Progress, Media and Custom templates; updates at most once per second; the system owns all Halo animation.
- **Control Center tiles:** a small toggle or slider an app can offer.

Developer workflow

```
sp new app --template list my-app          # scaffold from a template
cargo run -p my-app                       # run against the windowed compositor
on the laptop
sp package my-app --sign ~/.sphatik/dev.key
sp install my-app.spk --device             # push to the phone over USB
```

Distribution: the Sphatik Store for native apps, with review and signing; sideloading allowed after a one-time system warning; other stores can be added as sources, as the design spec requires.

Android compatibility layer and microG

Android apps run inside a full Android system in a lightweight container that shares Sphatik's kernel, using the same approach as Waydroid. They run natively on the Snapdragon 720G, with no emulation, and appear to the user as ordinary Sphatik apps.

How it is built

Piece	Choice
Container	LXC, unprivileged where possible, managed by Sphatik's container manager
Android system image	LineageOS for arm64 (Android 13 generation), built by us with the signature-spoofing patch microG needs, 32-bit ARM libraries included for older apps
Graphics	Mesa (Freedreno) and minigbm inside the container, so Android apps use the same GPU driver as Sphatik
Binder	The host kernel's binderfs, one instance per container

Piece	Choice
Audio	Android audio routed into PipeWire
Network	Virtual bridge with NAT through the host's connection; respects Sphatik's per-app network permission at the container level
Windows	Multi-window mode: every Android app is its own Wayland window, so it gets gestures, the app switcher and the glass status bar

Integration with Sphatik (what makes Android apps feel native)

Feature	How
App icons	The container manager lists installed Android packages and registers them with <code>org.sphatik.Apps</code> ; they appear on Home and in the App Library, marked with a small Android badge in Settings only
Notifications	<code>sphatik-bridge</code> , a small APK inside the container, listens to Android notifications and forwards them to Sphatik's Notification Center; replies and actions travel back
Clipboard	Synced both ways
Files	A shared folder: Sphatik's <code>Android</code> folder appears inside Android as <code>/sdcard/Sphatik</code>
Links	Opening a link can target a native or an Android app, following the user's default-app choice
Permissions	Camera, microphone and location go through Sphatik's prompts first, then Android's own runtime prompt
Keyboard	v1 uses an Android keyboard inside the container; v2 adds a bridge so Sphatik's keyboard types into Android apps

microG and app stores

- **microG** (GmsCore plus its companion services) replaces Google Play Services: push notifications through Firebase Cloud Messaging, Google account sign-in, maps and location APIs for apps that need them. Network location uses a configurable backend.
- **Aurora Store** downloads Play Store apps from Google's servers, anonymously or with the user's own Google account. **F-Droid** covers open-source apps.
- **Neither Google Play Services nor the Play Store is ever shipped in Sphatik.** The user installs Aurora Store themselves during Android setup.

What users can expect

App type	Expectation
Messaging and social (WhatsApp, Telegram, Instagram)	Generally work; push notifications depend on microG staying awake (see below)
Video and music (YouTube, Spotify)	Generally work; sign-in relies on microG and can be less reliable
Browsers, productivity, most games	Generally work
Banking, UPI payments, some games with anti-cheat	Often refuse to run: they check Google's Play Integrity, which a custom system cannot pass

Resource management on 4 GB

- The container starts on the first Android app launch (a few seconds cold) and uses roughly 800 MB to 1 GB of RAM.
- 30 seconds after the last Android window closes, it is frozen with the cgroup freezer, using no CPU.
- Under memory pressure it is stopped completely.
- Trade-off the user controls in Settings: **"Keep Android apps connected"** keeps a minimal part awake so WhatsApp messages arrive instantly, at a cost in RAM and battery; off by default, suggested when the user installs a messaging app.

Setup flow for the user: Settings → Android apps → Set up. Sphatik downloads the Android image (about 1.5 GB), starts it once, installs microG and the bridge, then offers Aurora Store and F-Droid with a one-screen explanation of what works and what does not.

Security and privacy

Sphatik protects user data with a verified system, encrypted storage and sandboxed apps, and is honest about the limits that come from running on an unlocked consumer phone.

Threat model

Threat	Defence	Residual risk
Malicious or buggy app	Sandbox (namespaces, seccomp, Landlock), permissions, portals	Kernel exploits that escape namespaces
Stolen phone, screen locked	Personal data encrypted with a key derived from the PIN; USB data off while locked	PIN guessing is rate-limited in software only (no hardware keystore on mainline)
Tampered system files	Read-only system image checked with dm-verity against Sphatik's signing key	A physical attacker can replace the boot image, because Xiaomi does not let a custom OS relock the bootloader
Network snooping	TLS everywhere in system services; optional encrypted DNS	Apps that ignore TLS best practice
Malicious update	Updates signed offline; the device rejects unsigned or older images	Compromise of the signing key (kept offline, see Updates)
Android app abuse	Container isolation, host-level permission gate, no Google services with system privilege	Android apps can still misuse data the user grants them

Encryption: the data partition holds a device-encrypted area (available before first unlock, for alarms and incoming-call ringing) and a credential-encrypted home (fscrypt, key from the PIN via Argon2id with a deliberately high cost). The key is wiped from memory when the phone reboots; after that, nothing personal is readable until the PIN is entered.

Secrets: apps store passwords and tokens through an `org.sphatik.Secrets` vault encrypted with the credential key; apps never see each other's secrets.

Privacy defaults

- No analytics or telemetry of any kind. Crash reports stay on the phone unless the user shares one.

- Status-bar dots for camera (green), microphone (orange) and location (blue), drawn by the compositor so apps cannot hide them.
- Clipboard-read toasts, a 7-day permission access log, one-tap revoke.
- Per-app network toggle in v2, enforced with a cgroup-aware firewall.

Developer access: SSH over USB only in Developer mode, only with a key the user adds, and only while unlocked.

Known limitations, stated to users in Settings → Security: unlocked bootloader; no hardware-backed key storage; no tap-to-pay (NFC and secure element unsupported); fingerprint unavailable until the mainline driver works.

Vulnerability handling: a published security contact and PGP key; acknowledgement within 72 hours; fixes for critical issues shipped as an out-of-band update within 14 days.

Updates, releases and signing

Updates install in the background into the inactive slot, take effect on the next restart, and roll back on their own if the new version fails to boot.

A/B layout on userdata: two system image slots (about 3 GB each, per the storage budget), a small boot-control file recording the active slot and remaining boot attempts, and the kernel plus initramfs for each slot. Because the boot partition holds only one image, the updater also rewrites `boot` last, after the new system image has been written and verified.

Update flow

1. sphatikd checks the update server once a day on Wi-Fi (or when the user taps Check).
2. It downloads a signed manifest, then a binary delta against the running version (full image as fallback).
3. It writes the new image to the inactive slot and verifies its hash and signature.
4. It installs the matching boot image and sets the new slot to try, with three attempts.
5. The user restarts (or it happens overnight while charging).
6. On healthy boot, init marks the slot good; after three failures, the initramfs falls back to the old slot and the user sees a short notice.

Release channels

Channel	Who	Cadence
Nightly	Developers	Every night from <code>main</code> , unsigned for production
Beta	Testers who opt in	Every 2 weeks

Channel	Who	Cadence
Stable	Everyone	Monthly, plus out-of-band security fixes

Versioning: `YEAR.MONTH.PATCH` for the OS (for example 2027.03.1); semantic versions for SDK crates so app developers get compatibility guarantees.

Signing keys

Key	Signs	Kept
Release key	System images and update manifests	Offline, on a hardware security key; used only on a clean release machine
Store key	Sphatik Store catalogue and first-party apps	Offline, separate from the release key
Developer keys	Third-party apps	By each developer
Nightly key	Nightly images	CI secret; devices on Stable reject it

Release checklist: all CI green, device test suite passed on curtana, performance budgets met, changelog written, images signed on the offline machine, staged rollout (10% for 48 hours, then everyone), rollback tested.

Testing, performance and debugging

Every change is tested on the laptop in minutes, and every night on a real Redmi Note 9 Pro against the performance budgets.

Test layers

Layer	What it covers	Where it runs
Unit tests	Logic in every crate: settings, permission rules, spring maths, calculator parser	Laptop and CI, seconds
Component tests	UI widgets rendered off-screen and compared to golden images	CI with software rendering
Compositor tests	Headless compositor with scripted touch input: gestures, panels, app switcher	CI

Layer	What it covers	Where it runs
UI flow tests	Drive the shell and apps through the accessibility tree: unlock, open app, send message, start timer	CI and device
Device tests	Hardware checklist (calls, SMS, Wi-Fi, audio, sensors), suspend and resume, boot and rollback	Nightly on the device runner
Soak tests	8-hour standby drain, 1-hour scroll and app-switch loop, memory growth	Weekly on the device runner

Performance budgets checked automatically (from the design spec, for the Redmi Note 9 Pro)

Metric	Budget	Measured with
Dropped frames in shell transitions	Zero at 60 Hz	Presentation-time feedback from the compositor
Touch to first pixel	Under 20 ms	Input timestamp to scan-out timestamp
Glass cost per frame	Under 3 ms GPU	GPU timer queries in the glass pass
Cold boot to lock screen	Under 15 s	Boot trace from initramfs to first lock-screen frame
First-party app cold launch	Under 500 ms	Launch intent to first frame
Idle system RAM	750 MB or less	cgroup memory statistics
Standby drain	Under 1% per hour	8-hour soak test

A pull request that breaks a budget on the nightly device run is reverted or fixed the same day.

Debugging tools

- `sp_logs` : structured logs from every service over USB.
- `sp_perf_trace` : a Perfetto trace combining kernel scheduling (ftrace) and Sphatik's own spans from the `tracing` crate.
- **Compositor debug overlay** (Developer options): frames per second, frame-time graph, damaged regions flashed in colour, glass cost per panel.
- **GPU counters** through Freedreno's performance-counter tools.

- `gdb` over SSH for native code; `adb` into the Android container.
- Crash dumps (minidumps) collected locally and viewable in Settings → Developer.

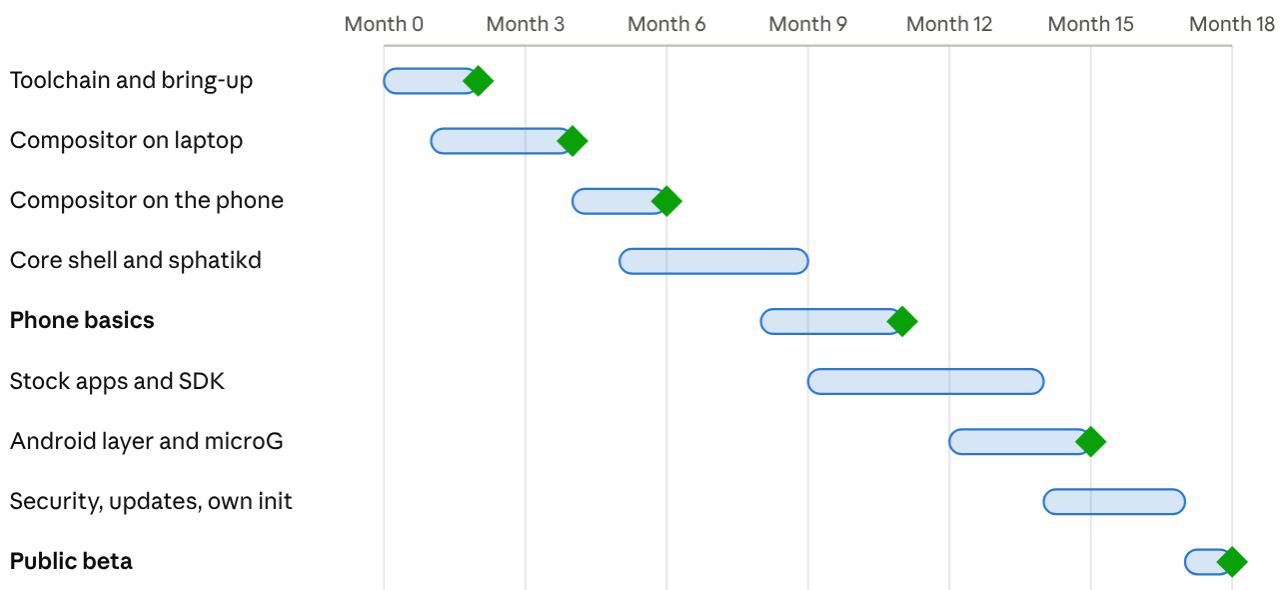
Device lab: one Redmi Note 9 Pro permanently connected to a self-hosted CI runner by USB, with a USB-controlled relay on its power button so tests can force reboots and recover a stuck device.

Roadmap and milestones

Nine overlapping phases take Sphatik from an empty repository to a public beta. Each gate must pass before the next dependent phase starts in earnest.

A usable Sphatik phone in about 11 months, public beta at 18

Months from start, one part-time developer; diamonds mark phase gates



Sphatik roadmap · 9 phases, 6 gates, 18 months

The estimate assumes one developer working part-time alongside college; a second contributor on apps or the Android layer would shorten it by roughly a third.

Gates

Gate	Pass criteria
Bring-up done	Base image boots on curtana; hardware checklist recorded; laptop toolchain builds and cross-compiles the workspace
Glass on laptop	Compositor with lock screen, home and one glass panel runs at 60 fps in a window; spring gestures feel right

Gate	Pass criteria
Glass on phone	Same shell runs full screen on curtana at 60 fps with working touch and under 3 ms glass cost
Usable phone	Calls, SMS, data, Wi-Fi, audio, notifications and camera work for a full week of daily use
Android apps	WhatsApp installs through Aurora Store, sends and receives messages, and its notifications appear in Notification Center
Public beta	Signed A/B updates with rollback, encrypted storage, all 16 stock apps, all performance budgets met

First concrete milestones (the next four weeks)

1. Create the monorepo, CI and the `sp` tool skeleton.
2. Run the windowed Smithay compositor on the laptop showing the Liquid Aurora wallpaper.
3. Draw one glass panel with the four-step blur and refraction shader.
4. In parallel: back up and unlock the Redmi, install the postmarketOS base and fill in the hardware checklist.

Licensing and legal

Sphatik is open source, ships no Google software, and meets the obligations of every licence it builds on. This section is an engineering checklist, not legal advice; get a qualified opinion before a public launch.

Sphatik's own code: Apache-2.0 for everything we write (OS, toolkit, SDK, apps). It is permissive for app developers, includes a patent grant, and is compatible with the GPL components we ship alongside.

Main dependencies and what they require

Component	Licence	What we must do
Linux kernel	GPL-2.0	Publish the exact kernel source and config for every image we ship
Mesa, Smithay, PipeWire, zbus, taffy	MIT or similar	Keep copyright notices in the NOTICE file

Component	Licence	What we must do
ModemManager, BlueZ, iwd	GPL or LGPL	Offer source for the versions we ship, including any patches
LineageOS (Android image)	Apache-2.0 with GPL parts	Publish our build recipe and patches; ship its notices
Waydroid tools (if reused)	GPL-3.0	Publish source of any modified parts
microG	Apache-2.0	Ship notices
Aurora Store, F-Droid	GPL-3.0	Not bundled; the user installs them
Fonts (Outfit during development, Sphatik Sans)	SIL Open Font License	Ship the licence; a modified font must use a different name
Qualcomm and Xiaomi firmware	Proprietary	Never redistributed; extracted from the user's own phone during install

Google and Android: no Google apps or Play Services are shipped or downloaded by Sphatik. Describing the feature as "runs Android apps" is fine; the Android name or robot logo must not appear in Sphatik's own branding.

Names: search the IP India trademark registry for "Sphatik" and "SKY" in the software and phone classes, and check domain and GitHub organisation availability, before any public release. "SKY" is a common word, so expect to need a distinctive logo mark.

Compliance automation: CI generates the NOTICE file and a software bill of materials (SBOM) for every image with a licence scanner, and fails the build if a dependency with an incompatible or unknown licence appears. A public page lists the source for every GPL component per release.

Privacy law: with no telemetry, the OS itself collects no personal data. The update server, Store accounts and crash reports the user chooses to send still need a privacy policy aligned with India's Digital Personal Data Protection Act, 2023.

Risks and open decisions

The biggest risks are in hardware support and scope, not in the UI; each has a named mitigation.

Risks

Risk	Likelihood	Impact	Mitigation
Mainline support for curtana stalls or regresses (modem, audio, camera)	Medium	High	Stay close to the community port; upstream fixes; keep a second supported device in mind (for example a OnePlus 6)
Call audio or modem reliability not good enough for daily use	Medium	High	Test early in Phase 0; the Usable phone gate blocks feature work until calls are reliable
Glass effects too heavy for Adreno 618	Low	Medium	Balanced tier, cached blur, quarter-resolution passes, automatic degradation order
4 GB RAM too tight with the Android container	Medium	Medium	Freeze or stop the container aggressively; zram; lazy start
Banking and UPI apps refuse to run	High	Medium	State it clearly in setup; recommend the bank's website as a fallback
Scope too large for one part-time developer	High	High	Gates, a strict v1 feature list, and recruiting contributors after the Glass on phone gate
Bricking the development phone	Low	High	Never touch persist or modem partitions; keep the stock fastboot ROM ready; spare phone for daily use

Open decisions (each gets an ADR when settled)

- ☐ App-side 2D renderer: `femtovg` for v1, or move straight to a Vulkan-based renderer later?
- ☐ Base userspace after bring-up: keep Alpine packages underneath, or build the full image with our own recipe?
- ☐ Store policy: review process, who can publish, and whether paid apps exist in v1.
- ☐ Sphatik Sans: draw from scratch or fork an OFL font and redraw its signature details (carried over from the design spec).
- ☐ Default for "Keep Android apps connected": off (battery) or on when a messaging app is installed (reliability).
- ☐ Second reference device to support after curtana, if any.